

# ■ DATABRICKS

Complete Reference Guide & Syntax with Code Examples

Apache  
Spark

Delta Lake

PySpark

SparkSQL

MLflow

DLT  
Pipelines

Streaming

Unity  
Catalog

■ Version

■ Databricks Runtime 14+

■ Language

Python (PySpark) | Spark SQL | Scala | R

■ Cloud

AWS | Azure | GCP

■ Topics

Spark Core • Delta Lake • DLT • MLflow • Streaming • SQL • Unity Catalog

# 1. Databricks Architecture Overview

Databricks is a unified analytics platform built on **Apache Spark**. It provides a cloud-based environment for big data processing, machine learning, and collaborative data engineering using notebooks, jobs, and pipelines.

| Layer         | Component                         | Description                                             |
|---------------|-----------------------------------|---------------------------------------------------------|
| Control Plane | Databricks UI / REST API          | Notebook editor, clusters, jobs, workspace management   |
| Data Plane    | Apache Spark Engine               | Distributed data processing (driver + executor nodes)   |
| Storage       | Delta Lake / DBFS / Cloud Storage | ACID-compliant data lake (S3, ADLS, GCS)                |
| ML Layer      | MLflow + Feature Store            | Experiment tracking, model registry, feature management |
| Governance    | Unity Catalog                     | Fine-grained access control across all data assets      |
| Ingestion     | Delta Live Tables (DLT)           | Declarative ETL pipeline framework with data quality    |

# 2. Cluster Configuration & Magic Commands

## 2.1 Cluster Types

- **All-Purpose Clusters** – Interactive notebooks, shared across users.
- **Job Clusters** – Ephemeral clusters created for automated jobs; cost-efficient.
- **SQL Warehouses** – Optimized for Databricks SQL analytics workloads.

## 2.2 Magic Commands

Use magic commands at the top of a notebook cell:

| Magic          | Description        | Example                                               |
|----------------|--------------------|-------------------------------------------------------|
| <b>%python</b> | Run Python code    | <code>%python<br/>print('Hello Databricks')</code>    |
| <b>%sql</b>    | Run Spark SQL      | <code>%sql<br/>SELECT * FROM my_table LIMIT 10</code> |
| <b>%scala</b>  | Run Scala code     | <code>%scala<br/>val df = spark.range(10)</code>      |
| <b>%r</b>      | Run R code         | <code>%r<br/>df &lt;- SparkR::sql('SELECT 1')</code>  |
| <b>%sh</b>     | Run shell commands | <code>%sh<br/>ls /dbfs/tmp</code>                     |

|                |                         |                              |
|----------------|-------------------------|------------------------------|
| <b>%fs</b>     | DBFS file operations    | %fs ls /                     |
| <b>%md</b>     | Render Markdown text    | %md<br># Heading<br>**bold** |
| <b>%run</b>    | Run another notebook    | %run ./utils/helper_notebook |
| <b>%pip</b>    | Install Python packages | pip install pandas==2.0.0    |
| <b>%conda</b>  | Use conda packages      | %conda install scikit-learn  |
| <b>%reload</b> | Reload imported module  | %reload mymodule             |

■ **Note:** Magic commands must be the first non-comment line in a cell. Only one magic per cell.

### 3. Apache Spark Fundamentals

### 3.1 SparkSession — Entry Point

```
# SparkSession is pre-created in Databricks notebooks as `spark`
from pyspark.sql import SparkSession

# Access the existing session
spark = SparkSession.builder \
    .appName("DatabricksDemo") \
    .config("spark.sql.shuffle.partitions", "200") \
    .config("spark.executor.memory", "4g") \
    .getOrCreate()

# Check version
print(spark.version)                # e.g. 3.5.0
print(spark.sparkContext.appName)   # "DatabricksDemo"
print(spark.sparkContext.defaultParallelism) # Number of cores

# Configure Spark at runtime
spark.conf.set("spark.sql.autoBroadcastJoinThreshold", -1)
spark.conf.get("spark.sql.shuffle.partitions") # "200"
```

## 3.2 Creating DataFrames

[illegible]

```
df6 = spark.range(1_000_000)           # one million rows
```

### 3.3 Reading Data (DataFrameReader)

[illegible]

### 3.4 Writing Data (DataFrameWriter)

```
# ■■ Save modes: overwrite | append | ignore | errorIfExists ██████████  
df.write \  
    .mode("overwrite") \  
    .option("header", "true") \  
    .csv("dbfs:/output/customers")  
  
# ■■ Write Parquet with partitioning ████████████████████████████████████  
df.write \  
    .mode("append") \  
    .partitionBy("year", "month") \
```

[illegible]

## 4. DataFrame Transformations & Actions

## 4.1 Column Operations

[illegible]

## 4.2 Aggregations & GroupBy

[illegible]

[illegible]

## 4.3 Joins

```
# ■■ Join types ████████████████████████████████████████████████████████
```

```
# inner, left (left_outer), right (right_outer), full (full_outer),
```

```
# left_semi, left_anti, cross
```

[illegible]

## 4.4 String & Date Functions

```
# String functions
F.upper("name"),      F.lower("name"),      F.initcap("name")
F.trim("col"),        F.ltrim("col"),        F.rtrim("col")
F.length("col")
F.substring("col", start=1, length=5)
F.concat("first", F.lit(" "), "last")
F.concat_ws("-", "y", "m", "d")      # concat with separator
F.split("full_name", " ")            # returns ArrayType
F.regexp_replace("col", r"^[a-zA-Z]", "")
F.regexp_extract("email", r"@(\w+)\.com", 1)
F.lpad("id", 8, "0")                  # left pad to width 8
F.rpad("id", 8, "0")                  # right pad
F.translate("col", "abc", "ABC")      # char-by-char replace
F.instr("col", "search_str")           # position of substring
F.locate("str", "col", 1)              # locate starting at pos
F.soundex("name")                     # phonetic encoding
F.levenshtein("s1", "s2")              # edit distance
F.format_string("%05d", F.col("num"))  # sprintf-style format
```

```
# Date / Timestamp functions
F.current_date()
F.current_timestamp()
F.to_date("date_str", "yyyy-MM-dd")
F.to_timestamp("ts_str", "yyyy-MM-dd HH:mm:ss")
F.date_format("date_col", "dd/MM/yyyy")
F.year("date"), F.month("date"), F.dayofmonth("date")
F.dayofweek("date"), F.dayofyear("date"), F.weekofyear("date")
F.hour("ts"), F.minute("ts"), F.second("ts")
F.date_add("date", 7) # add 7 days
F.date_sub("date", 30) # subtract 30 days
F.datediff("end_date", "start_date") # difference in days
F.months_between("end", "start") # months between
F.add_months("date", 3)
F.last_day("date") # last day of month
F.trunc("date", "month") # truncate to month
F.date_trunc("hour", "ts") # truncate timestamp
F.unix_timestamp("col", "yyyy-MM-dd") # to epoch seconds
F.from_unixtime(F.col("epoch"), "yyyy-MM-dd HH:mm:ss")
```

## 5. Spark SQL & Databricks SQL

## 5.1 Running SQL in Notebooks

```
# Method 1: SQL magic command (cell-level)
# Cell 
# %sql
# SELECT department, COUNT(*) AS emp_count, AVG(salary) AS avg_sal
# FROM employees
# WHERE hire_date >= '2020-01-01'
# GROUP BY department
# ORDER BY emp_count DESC

# Method 2: spark.sql() in Python
result = spark.sql("""
    SELECT
        dept,
        year(hire_date) AS hire_year,
        COUNT(*) AS headcount,
        ROUND(AVG(salary),2) AS avg_salary,
        MAX(salary) AS top_salary
    FROM employees
    WHERE status = 'active'
    GROUP BY dept, year(hire_date)
    HAVING COUNT(*) > 5
    ORDER BY hire_year DESC, headcount DESC
""")
result.show(truncate=False)

# Method 3: Register a temp view then query
df.createOrReplaceTempView("emp_view")
spark.sql("SELECT * FROM emp_view LIMIT 10").show()

# Global temp view (accessible across sessions)
df.createGlobalTempView("global_emp")
spark.sql("SELECT * FROM global_temp.global_emp")
```

## 5.2 DDL — Creating & Managing Tables

```
-- Create database / schema
CREATE DATABASE IF NOT EXISTS my_catalog.finance
  COMMENT "Finance department data"
  LOCATION "abfss://container@storage.dfs.core.windows.net/finance";

-- Create managed Delta table
CREATE TABLE IF NOT EXISTS my_catalog.finance.transactions (
  txn_id      BIGINT      NOT NULL,
  account_id  STRING      NOT NULL,
  amount      DECIMAL(18,2),
  currency    STRING      DEFAULT "USD",
  txn_date    DATE,
  created_at  TIMESTAMP    DEFAULT current_timestamp(),
```

```

        CONSTRAINT pk_txn PRIMARY KEY (txn_id)
    )
    USING DELTA
    COMMENT "All financial transactions"
    TBLPROPERTIES (
        "delta.autoOptimize.optimizeWrite" = "true",
        "delta.autoOptimize.autoCompact"   = "true",
        "delta.enableChangeDataFeed"       = "true"
    );

-- Create partitioned table
CREATE TABLE sales.partitioned_sales
    USING DELTA
    PARTITIONED BY (year INT, month INT)
    AS SELECT *, year(sale_date) AS year, month(sale_date) AS month
    FROM sales.raw_sales;

-- External table
CREATE TABLE IF NOT EXISTS external_logs
    USING PARQUET
    LOCATION "s3a://my-bucket/logs/"
    AS SELECT * FROM source_table;

-- CTAS (Create Table As Select)
CREATE OR REPLACE TABLE summary AS
    SELECT dept, SUM(sales) AS total FROM orders GROUP BY dept;

-- Alter table
ALTER TABLE employees ADD COLUMN manager_id BIGINT;
ALTER TABLE employees RENAME COLUMN mgr_id TO manager_id;
ALTER TABLE employees DROP COLUMN old_column;
ALTER TABLE employees SET TBLPROPERTIES ("my.key" = "value");
ALTER TABLE employees CHANGE COLUMN salary salary DOUBLE COMMENT "Base salary";

-- Drop objects
DROP TABLE IF EXISTS old_table;
DROP DATABASE IF EXISTS old_db CASCADE;
DROP VIEW IF EXISTS my_view;

```

## 5.3 DML — Insert, Update, Delete, Merge

```

-- INSERT INTO
INSERT INTO orders VALUES (1001, "A001", 250.00, "2024-01-15");
INSERT INTO orders SELECT * FROM staging_orders WHERE status = "new";

-- INSERT OVERWRITE (replaces entire table / partition)
INSERT OVERWRITE orders SELECT * FROM orders_v2;
INSERT OVERWRITE orders PARTITION (year=2024, month=1)
    SELECT * FROM orders WHERE year=2024 AND month=1;

-- UPDATE (Delta only)
UPDATE employees
    SET salary = salary * 1.10,
        updated_at = current_timestamp()

```

```

WHERE performance_rating = "Excellent";

-- DELETE (Delta only)
DELETE FROM orders WHERE status = "cancelled" AND order_date < "2023-01-01";

-- MERGE (upsert) – Delta only
MERGE INTO target AS t
USING source AS s
ON t.id = s.id
WHEN MATCHED AND s.status = "deleted" THEN DELETE
WHEN MATCHED THEN UPDATE SET
    t.name          = s.name,
    t.updated_at    = current_timestamp()
WHEN NOT MATCHED THEN INSERT (id, name, created_at)
    VALUES (s.id, s.name, current_timestamp())
WHEN NOT MATCHED BY SOURCE THEN DELETE; -- delete rows not in source

```

## 5.4 Advanced SQL — CTEs, Window, PIVOT

```

-- Common Table Expressions (CTE)
WITH monthly_sales AS (
    SELECT
        region,
        DATE_TRUNC("month", sale_date) AS month,
        SUM(amount) AS total_sales
    FROM sales
    GROUP BY region, DATE_TRUNC("month", sale_date)
),
ranked AS (
    SELECT *,
        RANK() OVER (PARTITION BY region ORDER BY total_sales DESC) AS rnk
    FROM monthly_sales
)
SELECT * FROM ranked WHERE rnk <= 3;

-- Window functions in SQL
SELECT
    employee_id,
    department,
    salary,
    AVG(salary)      OVER (PARTITION BY department)      AS dept_avg,
    RANK()           OVER (PARTITION BY department
                          ORDER BY salary DESC)          AS salary_rank,
    SUM(salary)      OVER (PARTITION BY department
                          ORDER BY hire_date
                          ROWS BETWEEN UNBOUNDED PRECEDING
                          AND CURRENT ROW)                AS running_total,
    LAG(salary, 1)   OVER (ORDER BY hire_date)            AS prev_salary,
    LEAD(salary, 1)  OVER (ORDER BY hire_date)            AS next_salary,
    FIRST_VALUE(salary) OVER (PARTITION BY department
                             ORDER BY salary DESC)       AS top_salary
FROM employees;

-- PIVOT

```

```
SELECT * FROM (
    SELECT year, quarter, revenue FROM sales
)
PIVOT (
    SUM(revenue) FOR quarter IN ("Q1", "Q2", "Q3", "Q4")
);

-- LATERAL VIEW EXPLODE (for arrays)
SELECT id, tag
FROM articles
LATERAL VIEW EXPLODE(tags) t AS tag;

-- CASE WHEN
SELECT name,
    CASE
        WHEN score >= 90 THEN "A"
        WHEN score >= 80 THEN "B"
        WHEN score >= 70 THEN "C"
        ELSE "F"
    END AS grade
FROM students;
```

## 6. Delta Lake — ACID Transactions

Delta Lake is an open-source storage format built on Parquet that adds **ACID transactions**, **schema enforcement**, **time travel**, and **scalable metadata** handling to data lakes.

## 6.1 Delta Table Operations (Python)

```
from delta.tables import DeltaTable  
from pyspark.sql import functions as F  
  
DELTA_PATH = "dbfs:/mnt/delta/customers"  
  
# ■■■ Create Delta table from DataFrame ■■■  
df.write.format("delta").mode("overwrite").save(DELTA_PATH)  
  
# ■■■ Load Delta table ■■■  
dt = DeltaTable.forPath(spark, DELTA_PATH)  
df = dt.toDF() # get as DataFrame  
  
# ■■■ MERGE (upsert) ■■■  
updates_df = spark.table("staging_customers")  
  
dt.alias("target").merge(  
    updates_df.alias("source"),  
    "target.id = source.id"  
) .whenMatchedUpdate(set={  
    "name":      "source.name",  
    "email":     "source.email",  
    "updated_at": F.current_timestamp()  
}) .whenNotMatchedInsert(values={  
    "id":        "source.id",  
    "name":      "source.name",  
    "email":     "source.email",  
    "created_at": F.current_timestamp()  
}) .whenNotMatchedBySourceDelete() \  
.execute()  
  
# ■■■ Update ■■■  
dt.update(  
    condition = F.col("status") == "inactive",  
    set = {"status": F.lit("archived"), "updated_at": F.current_timestamp()}  
)  
  
# ■■■ Delete ■■■  
dt.delete(F.col("created_at") < F.lit("2020-01-01"))  
  
# ■■■ History ■■■  
dt.history().select("version","timestamp","operation","operationParameters") \  
.show(10, truncate=False)  
  
# ■■■ Time Travel
```

[illegible]

## 6.2 Schema Evolution & Enforcement

```
# Schema enforcement (default) – reject writes with extra/wrong columns
try:
    new_df.write.format("delta").mode("append").save(DELTA_PATH)
except Exception as e:
    print("Schema mismatch:", e)

# Schema evolution – mergeSchema allows adding new columns
new_df.write.format("delta") \
    .mode("append") \
    .option("mergeSchema", "true") \
    .save(DELTA_PATH)

# Overwrite schema completely
new_df.write.format("delta") \
    .mode("overwrite") \
    .option("overwriteSchema", "true") \
    .save(DELTA_PATH)

# Enable column mapping (for renaming/dropping columns)
spark.sql("""
    ALTER TABLE my_table
    SET TBLPROPERTIES (
        "delta.minReaderVersion" = "2",
        "delta.minWriterVersion" = "5",
        "delta.columnMapping.mode" = "name"
    )
""")

spark.sql("ALTER TABLE my_table RENAME COLUMN old_col TO new_col")
spark.sql("ALTER TABLE my_table DROP COLUMN unwanted_col")

# Change Data Feed (CDC)
spark.sql("""
```

```

    ALTER TABLE orders
    SET TBLPROPERTIES ("delta.enableChangeDataFeed" = "true")
    "")
# Read changes
changes = spark.read.format("delta") \
    .option("readChangeFeed", "true") \
    .option("startingVersion", 5) \
    .table("orders")
# _change_type: insert | update_preimage | update_postimage | delete

```

## 6.3 Delta SQL Syntax

```

-- Time travel in SQL
SELECT * FROM delta.`/mnt/delta/orders` VERSION AS OF 5;
SELECT * FROM delta.`/mnt/delta/orders` TIMESTAMP AS OF "2024-03-01";
SELECT * FROM orders@v5;           -- shorthand version syntax

-- Describe table details
DESCRIBE DETAIL orders;
DESCRIBE HISTORY orders;
DESCRIBE EXTENDED orders;

-- Optimize and compact
OPTIMIZE orders;
OPTIMIZE orders WHERE date >= "2024-01-01";
OPTIMIZE orders ZORDER BY (customer_id, order_date);

-- Vacuum
VACUUM orders RETAIN 168 HOURS DRY RUN;  -- preview files to delete
VACUUM orders RETAIN 168 HOURS;          -- actual cleanup

-- Clone table
CREATE TABLE orders_backup CLONE orders;           -- shallow
CREATE TABLE orders_backup DEEP CLONE orders;      -- deep (full copy)
CREATE TABLE orders_v2024 CLONE orders VERSION AS OF 10;  -- clone from version

-- Restore
RESTORE TABLE orders TO VERSION AS OF 5;
RESTORE TABLE orders TO TIMESTAMP AS OF "2024-01-01 00:00:00";

```

## 7. Delta Live Tables (DLT) Pipelines

Delta Live Tables is Databricks' declarative framework for building reliable, maintainable, and testable data processing pipelines with built-in data quality.

## 7.1 DLT Pipeline — Full Example

[illegible]

[illegible]

## 7.2 DLT Expectations Summary

| Decorator                                        | Behavior on Violation      | Use Case                   |
|--------------------------------------------------|----------------------------|----------------------------|
| <code>@dlt.expect(name, cond)</code>             | Track metric, keep row     | Monitoring / alerting only |
| <code>@dlt.expect_or_drop(name, cond)</code>     | Drop violating rows        | Strict data quality filter |
| <code>@dlt.expect_or_fail(name, cond)</code>     | Fail entire pipeline       | Critical data invariants   |
| <code>@dlt.expect_all(rules_dict)</code>         | Track all, keep all rows   | Multi-rule monitoring      |
| <code>@dlt.expect_all_or_drop(rules_dict)</code> | Drop row if ANY rule fails | Multi-rule strict filter   |
| <code>@dlt.expect_all_or_fail(rules_dict)</code> | Fail if ANY rule violated  | Multi-rule critical        |

## 8. Structured Streaming

## 8.1 Streaming Sources & Sinks

[illegible]

```
# ■■■ Output modes ■■■
# "append"    - only new rows added (no aggregations allowed unless watermark)
# "update"    - only rows changed since last trigger
# "complete"  - entire result table (aggregations)

# ■■■ Watermarking (handle late data) ■■■
windowed = events \
    .withWatermark("ts", "10 minutes") \
    .groupBy(
        F.window("ts", "5 minutes", "1 minute"), # window, slide
        "user_id"
    ) \
    .agg(F.sum("amount").alias("total"))

# ■■■ foreachBatch (custom sink logic) ■■■
def process_batch(batch_df, batch_id):
    batch_df.write.format("delta").mode("append").saveAsTable("my_table")
    # Send to external system, run transformations, etc.

query = events.writeStream \
    .foreachBatch(process_batch) \
    .option("checkpointLocation", "dbfs:/checkpoints/batch/") \
    .start()

# ■■■ Monitor streaming queries ■■■
for q in spark.streams.active:
    print(q.name, q.status, q.lastProgress)

query.awaitTermination()
query.stop()
```

## 9. MLflow — ML Experiment Tracking

[illegible]



## 10. dbutils — Databricks Utilities

## 10.1 File System (dbutils.fs)

```
# List files
dbutils.fs.ls("dbfs:/mnt/data/")
dbutils.fs.ls("s3a://my-bucket/prefix/")

# Copy / Move / Delete
dbutils.fs.cp("dbfs:/source/file.csv", "dbfs:/dest/file.csv")
dbutils.fs.cp("dbfs:/source/dir/", "dbfs:/dest/dir/", recurse=True)
dbutils.fs.mv("dbfs:/old/path/", "dbfs:/new/path/", recurse=True)
dbutils.fs.rm("dbfs:/tmp/file.csv")
dbutils.fs.rm("dbfs:/tmp/old_dir/", recurse=True)

# Create directory
dbutils.fs.mkdirs("dbfs:/mnt/data/new_dir/")

# Read/write small text files
dbutils.fs.put("dbfs:/tmp/test.txt", "Hello, Databricks!", overwrite=True)
content = dbutils.fs.head("dbfs:/tmp/test.txt", maxBytes=1000)
print(content)

# Mount cloud storage
dbutils.fs.mount(
    source          = "wasbs://container@storage.blob.core.windows.net",
    mount_point     = "/mnt/my_storage",
    extra_configs  = {
        "fs.azure.account.auth.type": "OAuth",
        "fs.azure.account.oauth.provider.type":
            "org.apache.hadoop.fs.azurebfs.oauth2.ClientCredsTokenProvider",
        "fs.azure.account.oauth2.client.id":
            dbutils.secrets.get("scope", "client-id"),
        "fs.azure.account.oauth2.client.secret":
            dbutils.secrets.get("scope", "client-secret"),
        "fs.azure.account.oauth2.client.endpoint":
            f"https://login.microsoftonline.com/{tenant_id}/oauth2/token",
    }
)
dbutils.fs.unmount("/mnt/my_storage")
dbutils.fs.refreshMounts()
```

## 10.2 Secrets & Widgets

[illegible]

[illegible]

## 10.3 Unity Catalog & SQL Warehouse

```
-- Unity Catalog hierarchy: Catalog → Schema (Database) → Table/View
```

```
-- Set catalog context
USE CATALOG my_catalog;
USE SCHEMA finance;
```

```
-- Create catalog
CREATE CATALOG IF NOT EXISTS analytics
  COMMENT "Analytics catalog";
```

```
-- Grants and permissions
GRANT CREATE, USAGE ON SCHEMA analytics.sales TO `data-engineers`;
GRANT SELECT ON TABLE analytics.sales.orders TO `analysts`;
GRANT ALL PRIVILEGES ON CATALOG analytics TO `admins`;
REVOKE SELECT ON TABLE orders FROM `temp_user`;
```

```
-- Show grants
SHOW GRANTS ON TABLE analytics.sales.orders;
SHOW GRANTS TO `data-engineers`;
```

```
-- Row-level security (dynamic view)
CREATE VIEW secure_orders AS
SELECT * FROM orders
WHERE
    is_member("admin-group")           -- admins see all
```

```
    OR region = current_user_attr("region"); -- others see own region

-- Column masking function
CREATE FUNCTION mask_ssn(ssn STRING)
RETURNS STRING
RETURN IF(is_member("hr-group"), ssn, "****-**- " || RIGHT(ssn, 4));

-- Apply column mask
ALTER TABLE employees
    ALTER COLUMN ssn SET MASK mask_ssn;

-- Tags for governance
ALTER TABLE orders SET TAGS ("pii" = "true", "domain" = "finance");
ALTER TABLE orders ALTER COLUMN email SET TAGS ("pii" = "true");
```

## 11. Performance Optimization

## 11.1 Photon Engine & Adaptive Query Execution

[illegible]

## 11.2 Delta Lake Optimization

```
-- Optimize small files (bin packing)
OPTIMIZE my_table;
OPTIMIZE my_table WHERE date >= current date() - 7;
```

```

-- Z-Ordering (co-locate related data)
OPTIMIZE orders ZORDER BY (customer_id, order_date);
-- Improves query speed when filtering on Z-order columns

-- Liquid Clustering (replaces static partitioning – DBR 13+)
CREATE TABLE clustered_orders
CLUSTER BY (customer_id, order_date)
AS SELECT * FROM orders;

ALTER TABLE clustered_orders CLUSTER BY (customer_id, region);

-- Auto-optimize properties
ALTER TABLE my_table SET TBLPROPERTIES (
    "delta.autoOptimize.optimizeWrite" = "true", -- optimize on write
    "delta.autoOptimize.autoCompact"   = "true", -- compact after write
    "delta.targetFileSize"              = "134217728" -- 128MB target
);

-- Data skipping (automatic with Delta stats)
-- Delta stores min/max stats for first 32 columns
-- Filter on leading columns for best performance
SELECT * FROM orders WHERE order_date = "2024-01-15"; -- fast with stats

-- Bloom filter index
CREATE BLOOMFILTER INDEX ON TABLE orders
FOR COLUMNS (email OPTIONS (fpp=0.1, numItems=1000000));

```

## 11.3 Useful System Commands

```

-- Information schema queries
SELECT * FROM information_schema.tables
WHERE table_schema = "finance" AND table_type = "MANAGED";

SELECT * FROM information_schema.columns
WHERE table_name = "orders";

SELECT * FROM information_schema.table_privileges;

-- Table size and stats
DESCRIBE DETAIL orders;
-- Shows: numFiles, sizeInBytes, partitionColumns, etc.

SELECT
    table_name,
    ROUND(SUM(size) / 1e9, 2) AS size_gb,
    COUNT(*)                 AS num_files
FROM (DESCRIBE DETAIL my_table) t
CROSS JOIN LATERAL (SELECT explode(files)) f;

-- Show create table DDL
SHOW CREATE TABLE orders;

-- List all tables in schema
SHOW TABLES IN my_catalog.finance;

```

```
SHOW TABLES LIKE "order*";

-- Run shell commands (in notebook)
-- %sh
-- df -h
-- ls -la /dbfs/mnt/

-- DBFS utility commands
-- %fs ls /mnt/data/
-- %fs du /mnt/data/large_folder/
-- %fs head /mnt/data/sample.json
```

## 12. Pandas API on Spark (pandas-on-Spark)

Use the familiar pandas API with the power of distributed Spark computing. Available as `pyspark.pandas` (formerly Koalas).

[illegible]

```

# Iterator UDF (for large models / shared state)
from typing import Iterator
@pandas_udf("double")
def score_batch(batch: Iterator[pd.DataFrame]) -> Iterator[pd.Series]:
    model = load_model()          # load once per partition
    for df in batch:
        yield model.predict(df)

# Grouped map (transformation per group)
from pyspark.sql.functions import PandasUDFType
@pandas_udf(schema, PandasUDFType.GROUPED_MAP)
def normalize_by_group(pdf: pd.DataFrame) -> pd.DataFrame:
    pdf["norm"] = (pdf["value"] - pdf["value"].mean()) / pdf["value"].std()
    return pdf

result = df.groupBy("group").apply(normalize_by_group)

```

## 13. Advanced Patterns & Best Practices

## 13.1 UDFs (User Defined Functions)

[illegible]

## 13.2 Medallion Architecture Pattern

```
"""
Medallion Architecture: Bronze → Silver → Gold
████████████████████████████████████████████████████████████████████████████████
Bronze: Raw, unmodified data (schema-on-read)
Silver: Cleansed, validated, conformed (schema-on-write)
Gold:   Business-level aggregations, ML features, reporting
"""

CATALOG = "my_catalog"
DATABASE = "medallion"
```

[illegible]

## 13.3 Job Orchestration with Workflows



## 14. Quick Reference Cheat Sheet

### 14.1 Most-Used PySpark Functions

| Category    | Function                             | Description                   |
|-------------|--------------------------------------|-------------------------------|
| Aggregate   | F.count(*) / F.countDistinct(c)      | Row count / unique count      |
| Aggregate   | F.sum / F.avg / F.min / F.max        | Sum, average, min, max        |
| Aggregate   | F.first / F.last                     | First/last value in group     |
| Aggregate   | F.collect_list / F.collect_set       | Aggregate to array/set        |
| String      | F.upper / F.lower / F.initcap        | Case conversion               |
| String      | F.trim / F.split / F.concat_ws       | Clean / split / join strings  |
| String      | F.regexp_replace / F.regexp_extract  | Regex operations              |
| Date        | F.to_date / F.to_timestamp           | Parse strings to date/ts      |
| Date        | F.date_add / F.datediff              | Date arithmetic               |
| Date        | F.date_format / F.year / F.month     | Extract / format dates        |
| Array       | F.explode / F.posexplode             | Expand array to rows          |
| Array       | F.array_contains / F.size            | Check / measure arrays        |
| Array       | F.flatten / F.array_distinct         | Flatten / dedupe arrays       |
| Struct      | F.struct / F.col(s.field)            | Create / access struct        |
| Map         | F.create_map / F.map_keys            | Create / query maps           |
| Window      | F.rank / F.dense_rank / F.row_number | Ranking functions             |
| Window      | F.lag / F.lead                       | Offset functions              |
| Window      | F.sum().over(win)                    | Running aggregates            |
| Conditional | F.when / F.otherwise                 | IF-THEN-ELSE logic            |
| Conditional | F.coalesce / F.isnull                | Null handling                 |
| Math        | F.round / F.ceil / F.floor           | Rounding functions            |
| Math        | F.abs / F.pow / F.sqrt / F.log       | Math operations               |
| JSON        | F.from_json / F.to_json              | Parse/serialize JSON          |
| JSON        | F.get_json_object                    | Extract JSON path value       |
| Misc        | F.lit / F.col / F.expr               | Literal / column / expression |
| Misc        | F.monotonically_increasing_id()      | Unique row IDs                |
| Misc        | F.broadcast(df)                      | Broadcast hint for joins      |

### 14.2 Common Actions (Trigger Execution)

| Action                                    | Description                                  |
|-------------------------------------------|----------------------------------------------|
| <code>df.show(n=20, truncate=True)</code> | Print first n rows to console                |
| <code>df.collect()</code>                 | Return all rows as Python list (■■ OOM risk) |
| <code>df.take(n) / df.first()</code>      | Return first n rows / first row              |
| <code>df.count()</code>                   | Count total rows                             |
| <code>df.toPandas()</code>                | Convert to pandas (small data only)          |
| <code>df.write.*()</code>                 | Write data (save action)                     |
| <code>df.foreach(func)</code>             | Apply function to each row                   |
| <code>df.summary()</code>                 | Descriptive statistics                       |
| <code>df.printSchema()</code>             | Print schema tree                            |
| <code>df.columns / df.dtypes</code>       | List column names / types                    |